

ISLAMIC UNIVERSITY OF TECHNOLOGY



SWE 4301

LECTURE 16

Topic: Encapsulation and Modularization Smells

March 15, 2025

PREPARED BY

Maliha Noushin Raida

Lecturer, Department of CSE

Contents

1	Encapsulation Smells	3
1.1	Deficient Encapsulation	4
1.1.1	Rationale	4
1.1.2	Potential Causes	5
1.1.3	Example	5
1.2	Leaky Encapsulation	5
1.2.1	Rationale	6
1.2.2	Potential Causes	6
1.3	Missing Encapsulation	7
1.3.1	Rationale	7
1.3.2	Potential Causes	8
1.4	Unexploited Encapsulation	8
1.4.1	Rationale	8
1.4.2	Potential Causes	9
2	Modularization Smells	10
2.1	Broken Modularization	12
2.2	Potential Causes	12
2.3	Insufficient Modularization	13
2.3.1	Potential Causes	14
2.4	Cyclically dependent Modularisation	15
2.4.1	Example	16
2.5	Hub-Like Modularisation	16
2.5.1	Potential Causes	16
2.5.2	Example	17

1 ENCAPSULATION SMELLS

The principle of encapsulation advocates separation of concerns and information hiding through techniques such as hiding implementation details of abstractions and hiding variations.

Consider the example of a car. Do you need to know exactly how the engine of the car works in order for you to be able to drive it? Is it really required for you to know how the anti-braking system (ABS) of your car works? Well, you may know these details (if you are an automobile engineer), but these details are not required for you to drive the car.

This is precisely what the principle of Encapsulation does—it hides details that are not really required for the user of the abstraction (the abstraction in this case is the car). In addition, the principle of Encapsulation helps an abstraction to hide variation in the implementation details. For instance, whether your car has a petrol engine or a diesel engine, it does not change the way you drive your car.

The principle of encapsulation complements the principle of abstraction through information hiding. Encapsulation disallows (or “hides”) the users of the abstraction from seeing internal details of the abstraction.

Hence, there are two techniques that enable effective application of the principle of encapsulation:

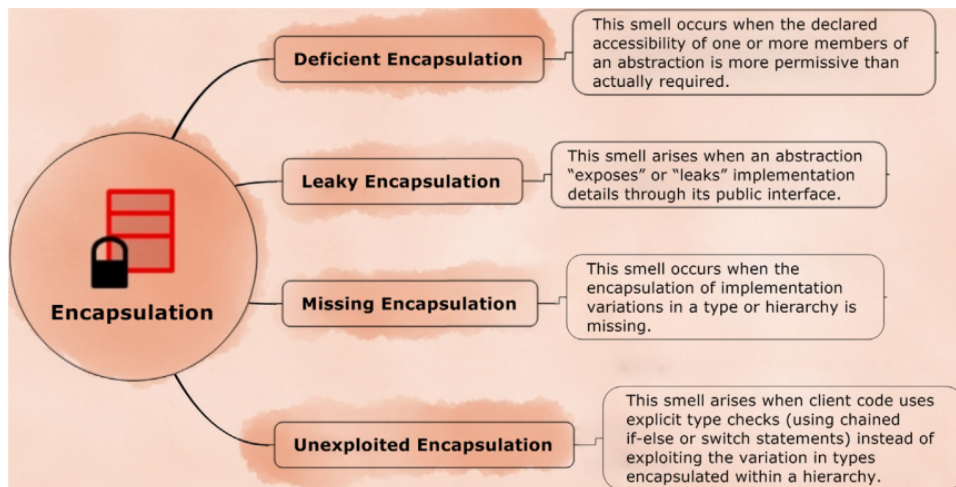
- **Hide implementation details**

An abstraction exposes to its clients only “what the abstraction offers” and hides “how it is implemented.” that is, the details of the implementation include the internal representation of the abstraction (e.g., data members and data structures that the abstraction uses) and details of how the method is implemented (e.g., algorithms that the method uses).

- **Hide variations**

Hide implementation variations in types or hierarchies. With such variations hidden, it is easier to make changes to the abstraction’s implementation without much impact to the clients of the abstraction.

All of the smells described in this chapter map to a violation of one of the above enabling techniques.

**Figure 1:** Encapsulation Smells

Design Smells	Violated Enabling Technique
Deficient Encapsulation, Leaky Encapsulation	Hide implementation details
Missing Encapsulation, Unexploited Encapsulation	Hide variations

Table 1: Design Smells and the Violated Encapsulation Enabling Techniques

1.1 Deficient Encapsulation

This smell occurs when the declared accessibility of one or more members of an abstraction is more permissive than actually required. For example, a class that makes its fields public suffers from Deficient Encapsulation. An extreme form of this smell occurs when there exists a global state (in the form of global variables, global data structures, etc.) that is accessible to all abstractions throughout the software system.

1.1.1 Rationale

implementation, which enables the two to change nearly independently. This separation of concerns allows the implementation details to be hidden from the clients who must depend only on the interface of the abstraction. If an abstraction exposes implementation details to the clients, it leads to undesirable coupling between the abstraction and its clients, which will impact the clients whenever the abstraction needs to change its implementation details. Providing more access than required can expose implementation details to the clients, thereby, violating the “principle of hiding.”

Having global variables and data structures is a more severe problem than providing lenient access to data members. This is because global state can be accessed and modified by

any abstraction in the system, and this creates secret channels of communication between two abstractions that do not directly depend on each other. Such globally accessible variables and data structures could severely impact the understandability, reliability, and testability of the software system.

Since this smell occurs when the internal state of an abstraction is inadequately hidden, we name this smell Deficient Encapsulation.

1.1.2 Potential Causes

- **Easier testability:** Often developers make private methods of an abstraction public to make them testable. Since private methods concern implementation details of an abstraction, making them public compromises the encapsulation of the abstraction.
- **Procedural thinking in object oriented context:** Sometimes, this smell occurs when developers from a procedural programming background using an object-orientated paradigm expose data as global variables when the data needs to be used by multiple abstractions.
- **Quick-fix solutions:** Delivery pressures often force developers to take shortcuts (which happens to be one of the main causes of technical debt). For instance, in situations where the data is shared between only a couple of abstractions, developers find it easier to expose that data globally than to create an abstraction and pass it as a parameter to a method.

1.1.3 Example

```
1 class BankAccount {  
2     public double balance; // Bad practice: balance should be private  
3  
4     public BankAccount(double balance) {  
5         this.balance = balance;  
6     }  
7 }
```

1.2 Leaky Encapsulation

This smell arises when an abstraction “exposes” or “leaks” implementation details through its public interface. Since implementation details are exposed, it not only harder to change the

implementation but also allows clients to directly access the internals of the object (leading to potential state corruption).

It should be noted that although an abstraction may not suffer from Deficient Encapsulation (i.e., the declared accessibility of its members are as desired), it is still possible that the methods in the public interface of the abstraction may leak implementation details.

1.2.1 Rationale

When implementation details of an abstraction are exposed through the public interface (i.e., there is a violation of the enabling technique “hide implementation details”):

- Changes made to the implementation may impact the client code.
- Exposed implementation details may allow clients to get handles to internal data structures through the public interface, thus allowing clients to corrupt the internal state of the abstraction accidentally or intentionally.

1.2.2 Potential Causes

- Lack of awareness of what should be “hidden” It requires considerable experience and expertise to discern and separate the implementation and interface aspects of an abstraction from each other. Inexperienced designers often inadvertently leak implementation details through the public interface.
- Viscosity It requires considerable thought and effort to create “leak-proof” interfaces. However, in practice, due to project pressures and deadlines, often designers or developers resort to quick and dirty hacks while designing interfaces, which leads to this smell.

```
1  import java.util.ArrayList;
2  import java.util.List;
3  class Student {
4      private List<String> courses; // Internal list of courses
5      public Student() {
6          this.courses = new ArrayList<>();
7      }
8      // Leaky encapsulation: Returning a reference to the internal list
9      public List<String> getCourses() {
10         return courses;
11     }
```

```
12     public void addCourse(String course) {
13         courses.add(course);
14     }
15 }
16
17 public class Main {
18     public static void main(String[] args) {
19         Student student = new Student();
20         student.addCourse("Math");
21         // Directly modifying the internal list from outside the class
22         student.getCourses().add("Physics");
23         System.out.println(student.getCourses()); // Outputs: [Math,
Physics]
24         // Even worse: Clearing all courses from outside the class
25         student.getCourses().clear();
26         System.out.println(student.getCourses()); // Outputs: []
27     }
28 }
```

1.3 Missing Encapsulation

This smell occurs when implementation variations are not encapsulated within an abstraction or hierarchy. This smell usually manifests in the following forms:

- A client is tightly coupled to the different variations of a service it needs. Thus, the client is impacted when a new variation must be supported or an existing variation must be changed.
- Whenever there is an attempt to support a new variation in a hierarchy, there is an unnecessary “explosion of classes,” which increases the complexity of the design.

1.3.1 Rationale

The Open Closed Principle (OCP) states that a type should be open for extension and closed for modification. In other words, it should be possible to change a type’s behavior by extending it and not by modifying it (see case study below). When the variations in implementation in a type or a hierarchy are not encapsulated separately, it leads to the violation of OCP.

One of the enabling techniques for Encapsulation is “hide variations.” when a type (or hierarchy) fails to encapsulate the variation in the implementation, it implies that the

principle of encapsulation has been either poorly applied or not applied at all. Hence, we term this smell Missing Encapsulation.

1.3.2 Potential Causes

- Lack of awareness of changing concerns
- Lack of refactoring
- Mixing up concerns
- Naive design decisions

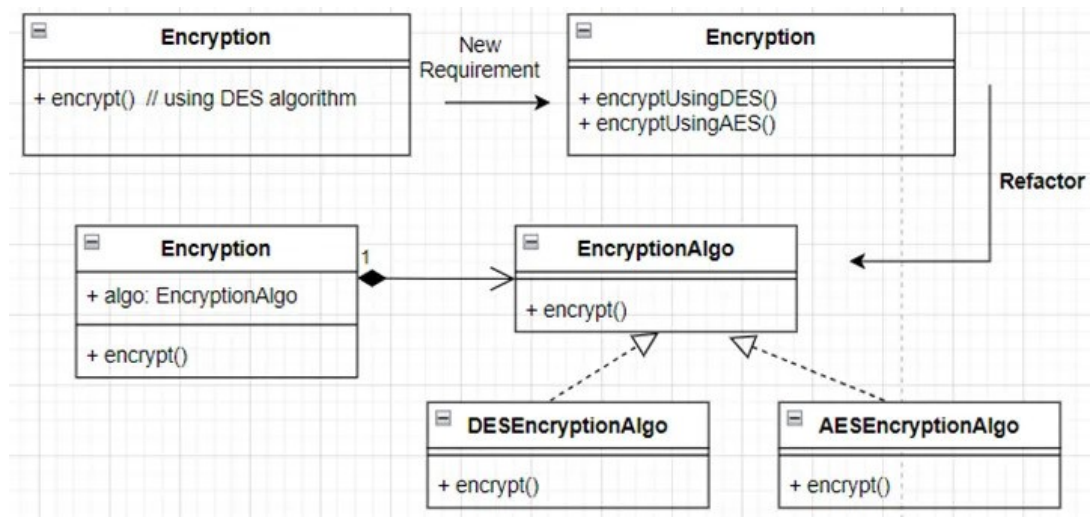


Figure 2: Example for Missing Encapsulation

1.4 Unexploited Encapsulation

This smell arises when client code uses explicit type checks (using chained if-else or switch statements that check for the type of the object) instead of exploiting the variation in types already encapsulated within a hierarchy.

1.4.1 Rationale

The presence of conditional statements (such as switch-case and if-else) in the client code that check a type explicitly and implement actions specific to each type is a well-known smell. Although this is a problem in general, consider the case when types are encapsulated in a

hierarchy but are not leveraged. In other words, explicit type checks are being used instead of relying on dynamic polymorphism.

Such explicit type checks are an indication of the violation of the enabling technique “hide variations” and the principle of “encapsulate what varies”. In the case of this smell, even though variation in types has been encapsulated within a hierarchy, it has not been exploited by the client code. Hence, we name this smell Unexploited Encapsulation.

1.4.2 Potential Causes

- **Procedural thinking in object-oriented language**

Developers from procedural background think in terms of encoding types and use switch or chained if-else statements for choosing behavior. When the same approach is applied in an object-oriented language, it results in this smell.

- **Lack of application of object-oriented principles**

Inexperienced software developers may be aware of object-oriented design principles, but they do not have a firm enough grasp of these concepts to actually put them into practice. For instance, they may be familiar with the concept of hierarchy and polymorphism, but may not be aware of how to exploit these concepts properly to create a high-quality design.

```
1 interface Shape {
2     void draw();
3 }
4 class Circle implements Shape {
5     @Override
6     public void draw() {
7         System.out.println("Drawing a Circle");
8     }
9 }
10 class Square implements Shape {
11     @Override
12     public void draw() {
13         System.out.println("Drawing a Square");
14     }
15 }
16 class DrawShape {
17     String shapetype;
18     DrawShape(String shapetype){
```

```
19 this.shapetype=shapetype;
20     }
21     public void draw() {
22         if(shapetype.equals('circle')) new Circle().draw();
23         else if(shapetype.equals('square')) new Square().draw();
24     }
25 }
26 //Rewriting DrawShape class
27 class DrawShape {
28     Shape shape;
29     DrawShape(Shape shape){
30         this.shape=shape;
31     }
32     public void draw() {
33         shape.draw();
34     }
35 }
```

2 MODULARIZATION SMELLS

Let us imagine that we rewrite a book and explain it like one long story without any breaks. The resulting book will not have chapters, sections, or subsections. There will be no cross referencing, no definite order to the discussion.

How would readers react to such a book? Would this be an easy read?

Similarly, it is important to follow the principle of modularization during software design. It should be pointed out that while modularization is generally considered to be a system-level concern (and concerns how we package abstractions to create logical modules), we use the term module here to mean abstractions at the class level, specifically, concrete classes, abstract classes, and interfaces. Thus, the goal behind modularization in the context of this book is to create cohesive and loosely coupled abstractions.

Enabling techniques that help apply the principle of modularization are,

- **Localize related data and methods**

Each abstraction should be cohesive in nature, i.e., the abstraction should keep related data and associated methods together.

- **Decompose abstractions to manageable size**

Break large abstractions into smaller ones that are moderate in size (i.e., neither too small, nor too large in size). For instance, a huge class not only makes it difficult for the reader to understand it, but also makes changes difficult because of possible interweaved responsibilities implemented by the class.

- **Create acyclic dependencies**

Abstractions should not be cyclically-dependent on each other. In other words, if a dependency graph is created, it should be free of cycles. Otherwise, a change in an abstraction may result in a ripple effect across the entire design.

- **Limit dependencies**

Create abstractions with low fan-in and fan-out. Fan-in refers to the number of abstractions that are dependent on a given abstraction. Thus, a change in an abstraction with high fan-in may result in changes to a large number of its clients. Fan-out refers to the number of abstractions on which a given abstraction depends. A large fan-out indicates that any change in any of these abstractions may impact the given abstraction. Thus, to prevent a ripple effect due to potential changes, it is important to reduce the number of dependencies between abstractions in the design.

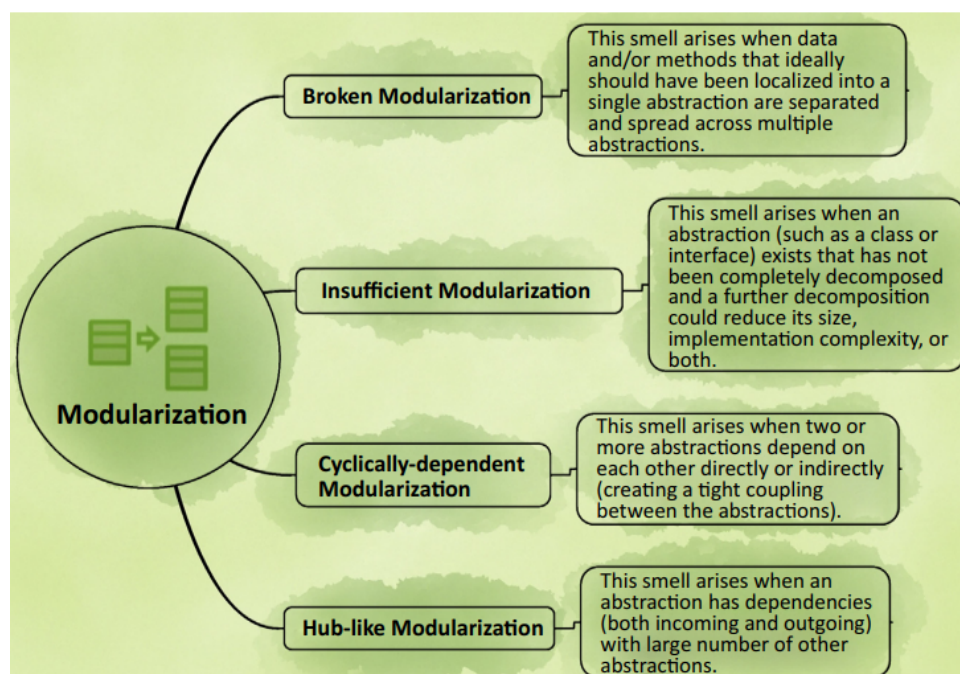


Figure 3: Modularization Smells

Design Smells	Violated Enabling Technique
Broken modularization	Localize related data and methods
Insufficient modularization	Decompose abstractions to manageable size
Cyclically-dependent modularization	Create acyclic dependencies
Hub-like modularization	Limit dependencies

Table 2: Design Smells and Corresponding Violated Enabling Technique

2.1 Broken Modularization

This smell arises when data and/or methods that ideally should have been localized into a single abstraction are separated and spread across multiple abstractions. This smell commonly manifests as:

- Classes that are used as a holder for data but have no methods operating on the data within that class.
- Methods in a class that are more interested in members of other classes.

2.2 Potential Causes

- **Procedural thinking in object-oriented languages**

Procedural languages provide language features such as “structs” (in C) and “record” (in Pascal) that hold data members together. Functions process the common data stored in structs/records. When developers from procedural language backgrounds such as C or Pascal move to an object-oriented language, they tend to separate data from functions operating on it, thereby resulting in this smell.

- **Lack of knowledge of existing design**

Large real-world projects have a complex design and have a codebase that is spread over numerous packages. In such projects, developers usually work on a small part of the system and may not be aware of the other parts of the design. Due to this, while implementing a new feature, developers may be unaware of the most appropriate classes where data/methods should be placed; this may lead to members being misplaced in the wrong classes.



Figure 4: Broken Modularization

2.3 Insufficient Modularization

This smell arises when an abstraction exists that has not been completely decomposed, and a further decomposition could reduce its size, implementation complexity, or both. There are two forms of this smell:

- Bloated interface: An abstraction has a large number of members in its public interface.
- Bloated implementation: An abstraction has a large number of methods in its implementation
-

2.3.1 Potential Causes

- **Providing centralized control** A typical cause of bloated implementation is centralizing control and assigning a large amount of work to a single abstraction or a method in an abstraction.

- **Creating large classes for use by multiple clients**

Often, software developers create a large abstraction so that it can be used by multiple clients. However, creating a single large abstraction that fits the needs of multiple clients leads to many problems, such as reduced changeability and extensibility of the design (see the Impacted Quality Attributes section).

- **Grouping all related functionality together**

Often, inexperienced developers tend to group together and provide all related functionality in a single class or interface without understanding how the Single Responsibility Principle (SRP) should be properly applied. This results in bloated interfaces or classes.

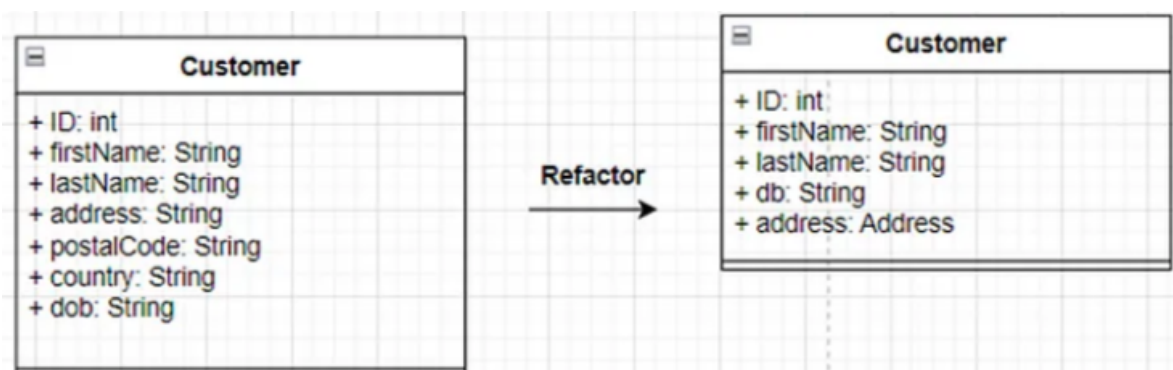


Figure 5: Insufficient Modularization Smell

If we rethink, a customer may have many other fields. With time, this class may end up having thousands of lines of code and become GOD class. So, this is an example of insufficient Modularisation. To refactor it, we need to identify highly cohesive code or attributes and move them out into a meaningful class or abstraction and make the Customer class use these abstractions by delegation. One such refactoring is to abstract address details into Address abstraction.

2.4 Cyclically dependent Modularisation

This smell arises when two or more abstractions depend on each other directly or indirectly (creating a tight coupling between the abstractions).

In the presence of cyclic dependencies, the abstractions that are cyclically-dependent may need to be understood, changed, used, tested, or reused together. Further, in case of cyclic dependencies, changes in one class (say A) may lead to changes in other classes in the cycle (say B). However, because of the cyclic nature, changes in B can have ripple effects on the class where the change originated (i.e., A). Large and indirect cyclic dependencies are usually difficult to detect in complex software systems and are a common source of subtle bugs.

- **Improper responsibility realization**

Often, when some of the members of an abstraction are wrongly misplaced in another abstraction, the members may refer to each other, resulting in a cyclic dependency between the abstractions.

- **Passing a self reference**

A method invocation from one abstraction to another often involves data transfer. If instead of explicitly passing only the required data, the abstraction passes its own reference (for instance, via “this”) to the method of another abstraction, a cyclic dependency is created.

- **Hard-to-visualize indirect dependencies**

In complex software systems, designers usually find it difficult to mentally visualize dependency relationships between abstractions. As a result, designers may inadvertently end up creating cyclic dependencies between abstractions.

2.4.1 Example

As we can see in below example, there are two service classes AService and BService doing different works have a circular dependency. The real question is why your conception making A and B dependent of each over? It could be that A and B represent the same concept and may be merged. So merging AService and BService to create a new service ABService is one of the refactoring solutions. It could also be that a part of your service A or B should be extracted in a new service C, which is a second refactoring solution.

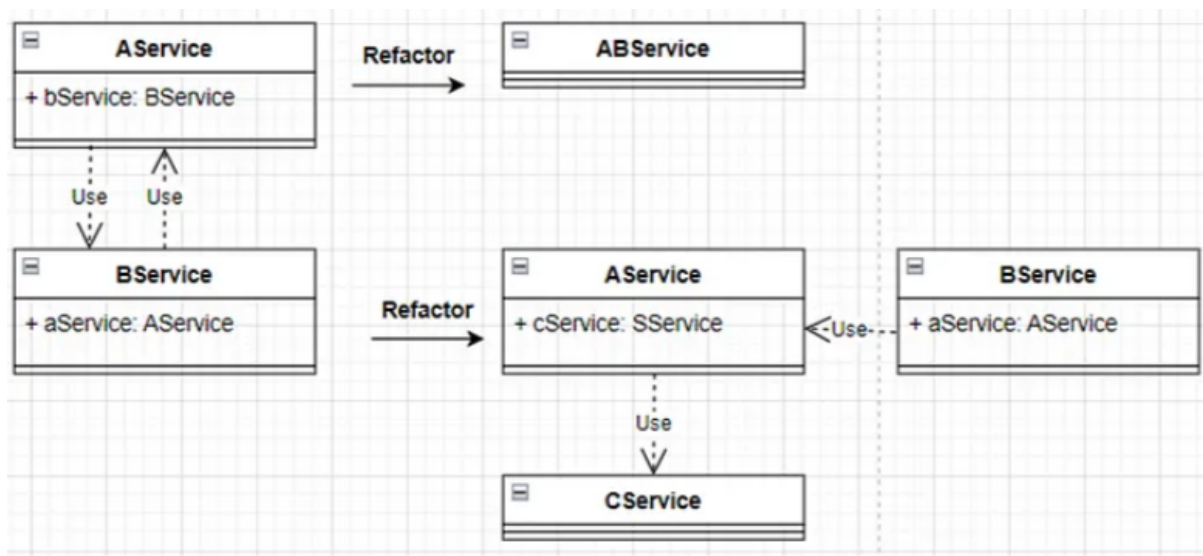


Figure 6: Cyclically dependent Modularization

2.5 Hub-Like Modularisation

This smell arises when an abstraction has dependencies (both incoming and outgoing) with a large number of other abstractions. “High cohesion and low coupling” is the basis for effective modularization. Meyer’s “few interfaces” rule for modularity says that “every module should communicate with as few others as possible.” For effective modularization, we must follow the enabling technique “limit dependencies.” When an abstraction has large number of incoming and outgoing dependencies, the principle of modularization is violated.

2.5.1 Potential Causes

- **Improper responsibility assignment (to a hub class)**

When an abstraction is overloaded with too many responsibilities, it tends to become a hub class with a large number of incoming as well as outgoing dependencies. In other words, most classes in the design will talk to this hub class, and this hub class also communicates with most other classes.

- **“Dump-all” utility classes**

Classes that provide supporting functionality often grow very large and get coupled with a large number of other classes. Consider the `javax.swing.SwingUtilities` class that is used by numerous other classes in the swing package. This is not surprising given the fact that it is a utility class. However, what is surprising is the number of UI classes in swing package it refers back to—such a large number of incoming and outgoing dependencies makes this class a hub class. In most cases utility classes often become common “dump-all-ancillary-functionality-here”

- classes that turn out to be hub classes.

2.5.2 Example

As we can see in below example, `java.util.Component` class has 498 incoming and 71 outgoing dependencies. This means this class is not following single responsibility principle. To refactor it, we need to split the responsibilities by creating new classes and move functionalities into there.

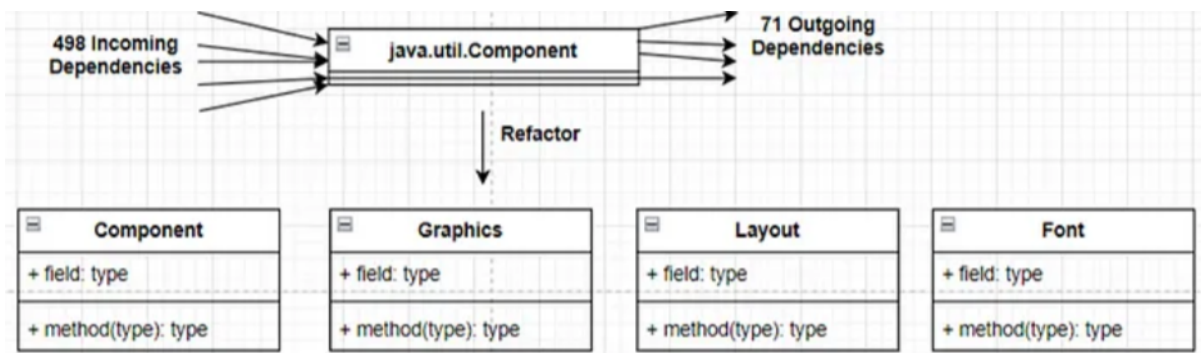


Figure 7: Hub-Like Modularization